PET ADOPTION SYSTEM

SYSTEM DOCUMENTATION

SUBMITTED BY,
Everlyne willy
DCS-03-8537/2024

A SYSTEM DOCUMENTATION SUBMITTED IN PARTIAL FULFILMENT FOR THE AWARD OF DIPLOMA IN COMPUTER SCIENCE BY ZETECH UNIVERSITY

APRIL 2026

DECLARATION

I declare that this project is my original work and has not been submitted to any other institution for academic credit. All source of information have been duly acknowledged.

Student name: Everlyne willy

Signature:

Date:

DEDICATION

I dedicate this project to my family for their support and encouragement throughout my studies. I also dedicate it to my friends for their motivations. And above all, I thank God for granting me wisdom and patience to complete this work.

ABSTRACT

This project focuses on the design and development of a pet adoption system aimed at improving efficiency , accuracy and convenience in managing adoption process. The system addresses common challenges such as manual record-keeping, difficult in tracking pet information and limited communication between shelters and potential adopters. It integrates key features including digital pet registration , online adoption applications , automated record updates and real-time availability status for each pet. The project follows waterfall model progressing through structured phases such as requirement analysis, system design, implementation, testing, deployment and maintenance. By automating essential shelter operations , pet adoption system seeks to reduce paperwork , minimize errors , speed up adoption process and enhance decision making through accurate digital records. Overall, the system improves the adoption experience for both shelter staff and adopters while promoting better care and management of animals.

DEFINITION OF KEY TERMS

Adopter- a registered user who applies to adopt a pet through the system.

Administrator- a system manager responsible for overseeing operations such as managing users, pets and adoption requests.

ABBREVIATIONS AND ACRONYMS

HTML – HyperText Markup Language

CSS – Cascading Style Sheets

JS – JavaScript

React.js – React JavaScript Library

PHP – Hypertext Preprocessor

MySQL – My Structured Query Language

API – Application Programming Interface

JWT – JSON Web Token

HTTPS – Hypertext Transfer Protocol Secure

IDE – Integrated Development Environment

XAMPP – Cross-Platform Apache, MySQL, PHP, Perl

cURL – Client URL Library

LIST OF FIGURES

Figure 1 6
Figure 2 7
Figure 3 7
Figure 4 8
Figure 5 8
Figure 6 ..... **Error! Bookmark not defined.**
Figure 7 8
Figure 8 10
Figure 9 11
Figure 10 12
Figure 11 13
Figure 12 14
Figure 13 15
Figure 14 16
Figure 15 16
Figure 16 18
Figure 17 19

LIST OF TABLES

Table 1 3
Table 2 5

TABLE OF CONTENTS

Contents

DECLARATION ii
DEDICATION iii
ABSTRACT iv
DEFINITION OF KEY TERMS v
ABBREVIATIONS AND ACRONYMS vi
LIST OF FIGURES vii
LIST OF TABLES ix
TABLE OF CONTENTS x
CHAPTER ONE: PROJECT PLANNING AND ANALYSIS 1
1.1 Statement of Problem 1
1.2 Study Justification 1
1.3 System Objectives 2
1.3.1 General Objective 2
1.3.2 Specific Objectives 2
1.4 Functional Requirements 3
1.5 Breakdown of Tools & Resources to Be Used 4
1.6 Project Schedule Breakdown 5
CHAPTER TWO: DESIGN AND MODELING 6
2.1 Introduction to Modelling 6
2.2 User Interface Models 6
2.2.1 Login page 6
2.2.2 Registration page 7
2.2.3 Screening page 7
2.2.4 Pet listing page 7
2.2.5 Pet details page 8
2.2.6Admin dashboard 8
2.3 Logic Models 9
2.3.1 Login process flowchart 9
2.3.2 Registration process flowchart 10
2.3.3 Screening process flowchart 11
2.3.4 Pet listing flowchart 12
2.3.5 Pet details 13

2.3.6 Payment 14
2.3.9 Entity-Relationship Diagram(ERD) 17
2.3.10 Data Flow Diagrams(DFD) 18

..... 19
CHAPTER THREE: SYSTEM IMPLEMENTATION 19
3.1 Introduction 19
3.2 User Interface Development 19
3.2.1 Login Page 20
3.2.2 Registration Page 24
3.2.3 Pet listing page 29
3.2.4 Pet Details Page 30
3.2.5 Screening form 33
3.2.6 Admin dashboard 37
3.3 Logic Development 39
3.3.1 Login validation 39
3.3.2 Registration validation 41
3.3.3 Pet Upload 41
3.3.4 Screening form processing 43
3.3.5 Admin approval 45
3.4 Testing 47
3.5 Deployment 47
CHAPTER FOUR: CONCLUSION AND RECOMMENDATION 48
4.1 Conclusion 48
4.2 Recommendation 48
REFERENCES 48

CHAPTER ONE: PROJECT PLANNING AND ANALYSIS

1.1 Statement of Problem

The problem of pet homelessness is being exacerbated due to huge numbers of animals being abandoned, relinquished, or running loose in the streets (Dog welfare project, 2023). Shelters and animal rescues take care of these animals for a while, but the rate of adoption is somewhat low , leading to overcrowding.

One contributing factor to the problem is the usage of conventional ways of pet adoption like word of mouth, going round picking up notices about pets for sale and rats for sale at a shelter etc. Those techniques are sluggish, reach only so far, and generally don't get enough eyes on the animals that need homes (Thakare, 2025)

Meanwhile, a lot of potential adopters have a hard time getting good information on what animals are available. They also might have to travel long distances to shelters, ask staffers multiple times for information about the dogs or work with incomplete information about them. This leads to missed opportunities in which potential adopters and pets do not find one another (Dog welfare project, 2023).

1.2 Study Justification

The development of pet adoption system is a suitable response to the challenges identified because it provides a centralized and structured platform for managing pet adoption activities (Thakare, 2025).The system allows shelters and administrators to register and upload complete and accurate details of available pets, ensuring that potential adopters can easily access up-to-date information in one place (Dog welfare project, 2023).

Through user registration and profile management , adopters can submit adoption requests online , reducing the need for time-consuming physical visits and informal communication. The system also includes adoption request tracking and status updates, which improves transparency and accountability for both shelters and adopters (Home page, 2022).

In addition , administrative features such as user management and pet status updates help reduce errors associated with manual record-keeping , improve operational efficiency and support faster placement of animals into suitable home

1.3 System Objectives

1.3.1 General Objective

(i) Development of Pet Adoption System

1.3.2 Specific Objectives

(i) To implement user registration and login for adopters and administrator to access the system.

(ii) To implement pet management module for adding, updating and viewing pets available for adoption.

(iii) To implement a screening form for collection and verification of adopter information before adoption approval.

(iv) To develop an adoption request management module that enables administrators to approve or reject adoption applications.

(v) To implement an email notification feature that informs adopters about the approval or rejection of their adoption requests.

1.4 Functional Requirements

TABLE 1

User	User Activities	Features
Adopter	- Register an account	- Online registration form
	- Log into the system	- Secure login and authentication
	- View available pets	- Pet listing page with images and details
	- Search and filter pets	- Search bar and filter by species, age, gender
	- Complete adopter screening form	- Adopter verification module
	- Receive email notification	- Email notification system
Administrator	- Log into the system	- Secure administrator login
	- Add new pets	- Pet management module
	- Approve or reject adoption requests	- Request management module
	- Review adopter details	- Adopter verification module

Table 1.4 Functional Requirements Table

1.5 Breakdown of Tools & Resources to Be Used

1. Software Tools
    - Fronted-End: HTML , CSS, JavaScript(React.js)- for structure , styling and interactivity.
    - Back-End: PHP – for system logic
    - Database: MySQL- for structured storage and retrieval of system data.
    - Security: HTTPS(encryption), JWT(authentication), bcrypt(password hashing)
2. Development Environment
    - IDE: Visual Studio Code
    - Local server: XAMPP(Apache, PHP, MySQL)
3. Deployment Resources
    - Netlify: free hosting for frontend
    - InfinityFree: free hosting for backend(PHP &MySQL)
4. Hardware Resources
    - Client devices: Computers and smartphones
    - Server machine : Local pc(XAMPP) or free-tier cloud server(infinityFree)
    - Network: internet connection(for accessing the system)
5. Human Resources
    - System developer: designs, codes and documents the system
    - Test Users: classmates for test usability and functionality
    - Supervisor: Guides, reviews and evaluates the project

1.6 Project Schedule Breakdown

TABLE 2

	PROJECT MILESTONES					
WEEKS	Project Planning & Analysis (System Documentation: Coverpage & Chapter One)	Project Design & Modeling (System Documentation Chapter Two)	Project Development & Testing (System Documentation Chapter Three)	Project Deployment (System Documentation Chapter Three)	Final Touches of System Documentation (Preliminary Pages, Chapter Four & References)	Project Presentation
7-9 Jan						
12-16 Jan						
19-23 Jan us						
26-30 Jan						
2-6 Feb						
9-13 Feb						
16-20 Feb						
23-27 Feb						
2-6 Mar						
9-13 Mar						
16-20 Mar						
23-27 Mar						
3rd April						

Table 1.6 Project Schedule Breakdown

CHAPTER TWO: DESIGN AND MODELING

2.1 Introduction to Modelling

In this chapter you'll find the designs and models used to visualize the system before development. These include diagrams and sketches that show how the system works and how users interact with it. Modelling the system first helped identify problems early, plan the development better and make the final system more organized and effective.

2.2 User Interface Models

2.2.1 Login page

The login page allows registered users to access the system by entering their username, email and password. It also provides links for password recovery and new user registration.

FIGURE 1

Figure 2.2.1 Login Form Design

2.2.2 Registration page

The registration page allows new users to create an account by entering their email, phone number, username, password and address. Users who already have an account can navigate to the login page.

FIGURE 2

figure2.2.2 Registration form design

2.2.3 Screening page

This form is used to collect information from adopters before approval . It includes questions about living conditions, experience with pets and suitability for adoption.

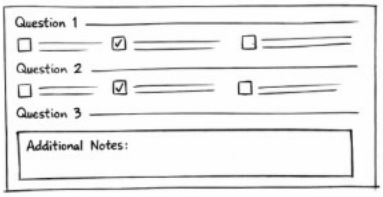


FIGURE 3

Figure 2.2.3 Screening form design

2.2.4 Pet listing page

This page displays all available pets with their images, names, breed , age and status. Users can view more details or apply for adoption.

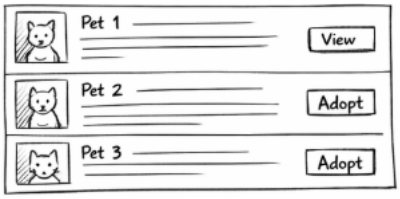


FIGURE 4

Figure 2.2.4 Pet listing form design

2.2.5 Pet details page

The pet details page shows full information about a selected pet including medical history, description and an adoption button.

FIGURE 5

Figure 2.2.5 Pet details form desig

2.2.6Admin dashboard

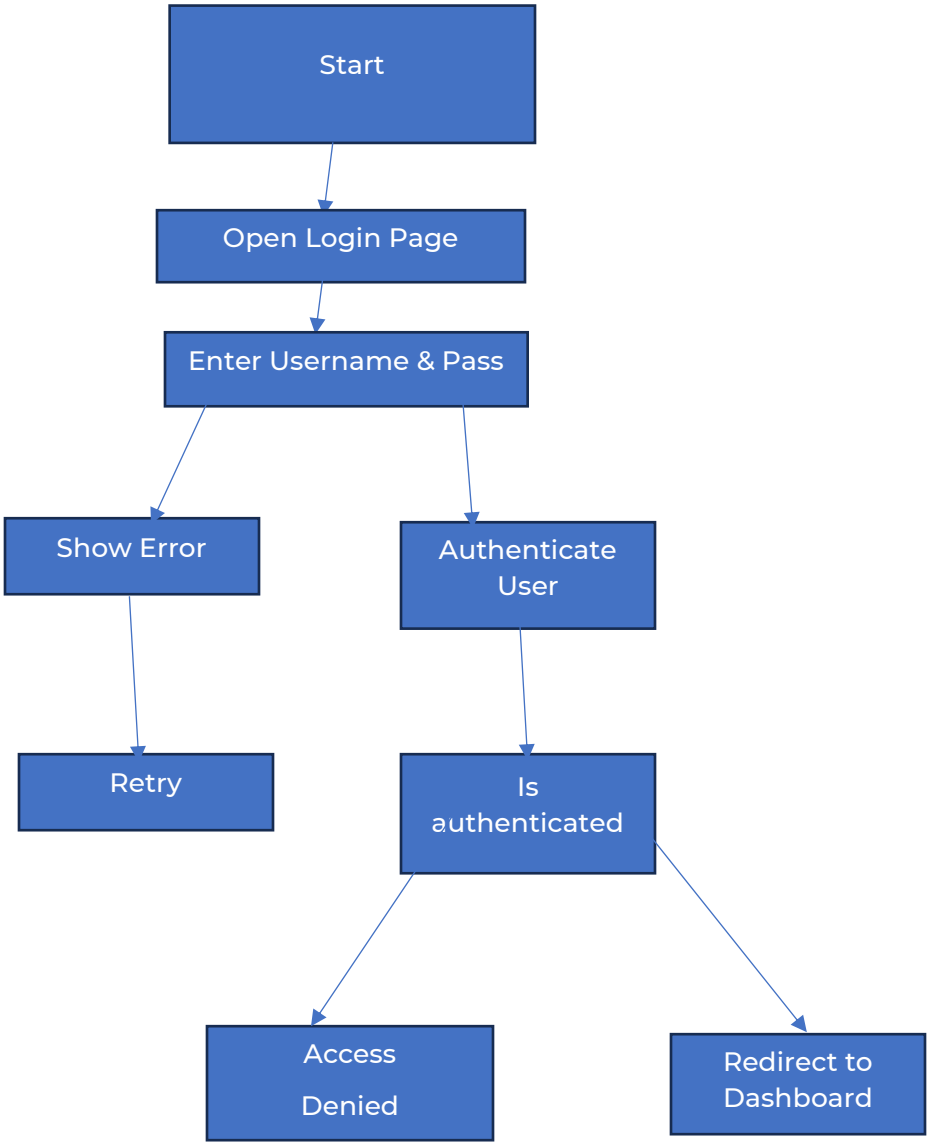
The admin dashboard allows administrators to manage users, pets, adoption requests and view system statistics.

FIGURE 6FIGURE 2.3.6 ADMIN DASHBOARD FORM DESIGN

2.3 Logic Models

2.3.1 Login process flowchart

The user enters login credentials which are sent to the system for validation. The system checks the credentials against the user database and returns a success or error message to the user. If the credentials are valid , the user is granted access to the system dashboard.



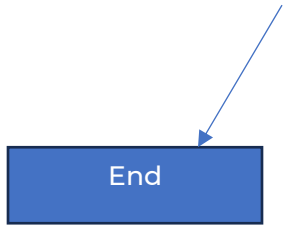
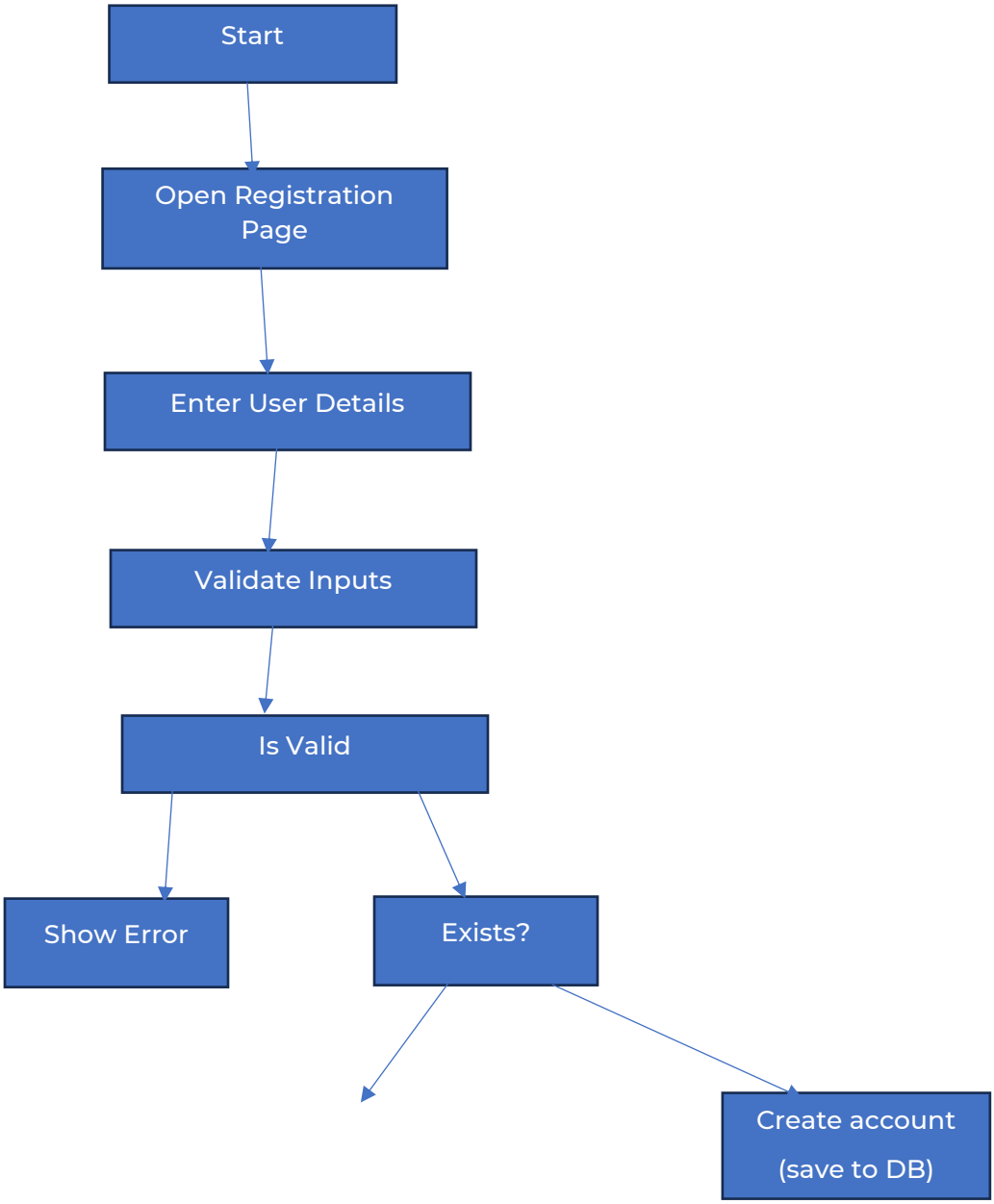


FIGURE 7

2.3.2 Registration process flowchart

The user enters registration details which are sent to the system for validation. The system validates the data and stores it in the user database. A confirmation message is then sent back to the user indicating successful registration.



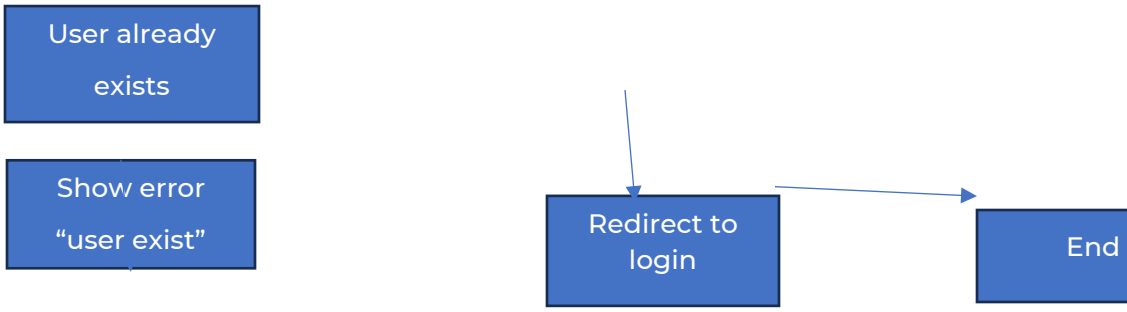
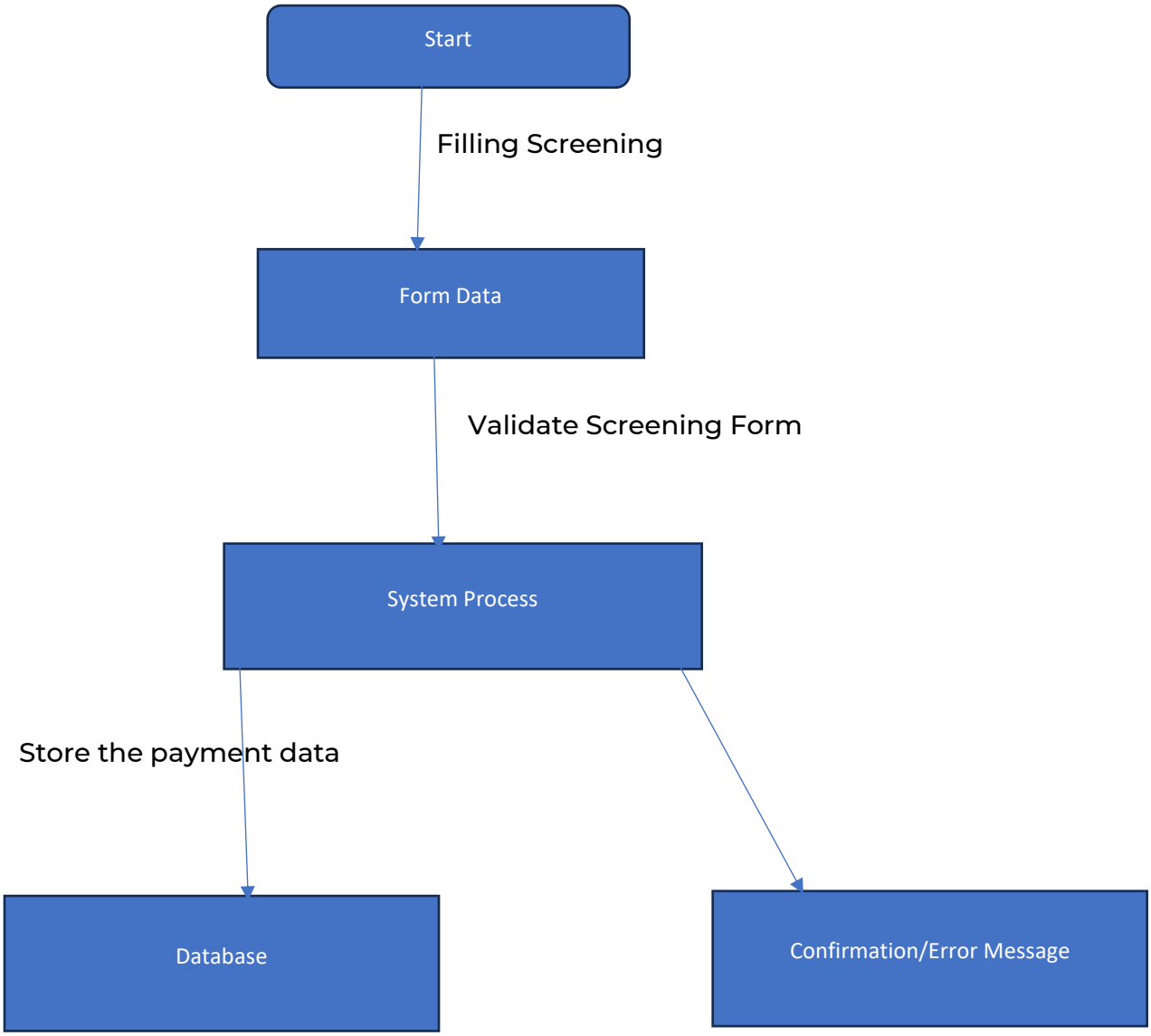


FIGURE 8

2.3.3 Screening process flowchart

The user fills in the screening form and submits the data to the system. The system validates the information and stores it in the screening database. The system then sends a confirmation or error message back to the user.



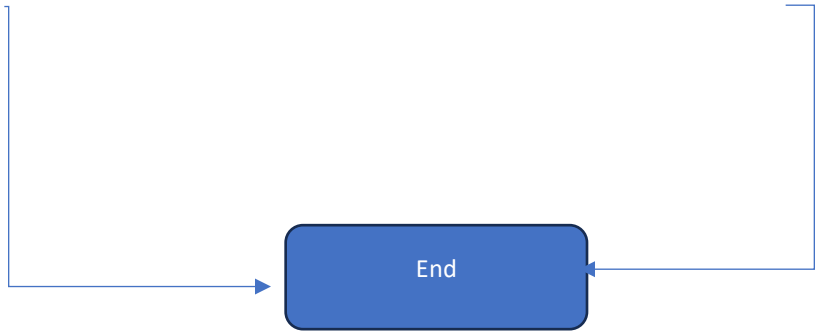


FIGURE 9

2.3.4 Pet listing flowchart

The user requests to view available pets. The system retrieves pet records from the pet database and displays the lists of pets to the user including images , breed , age and status.

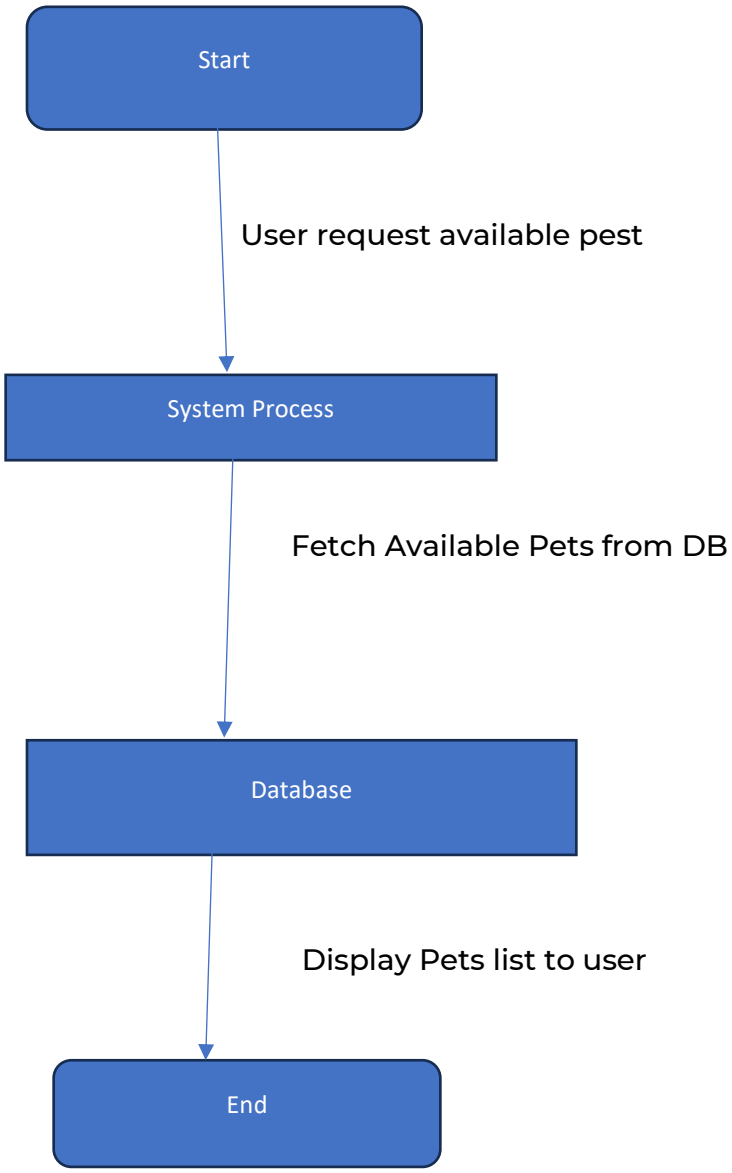


FIGURE 10

2.3.5 Pet details

The user selects a specific pet from listing. The system fetches detailed information about the selected pet from the pet database and displays the full details to the user.

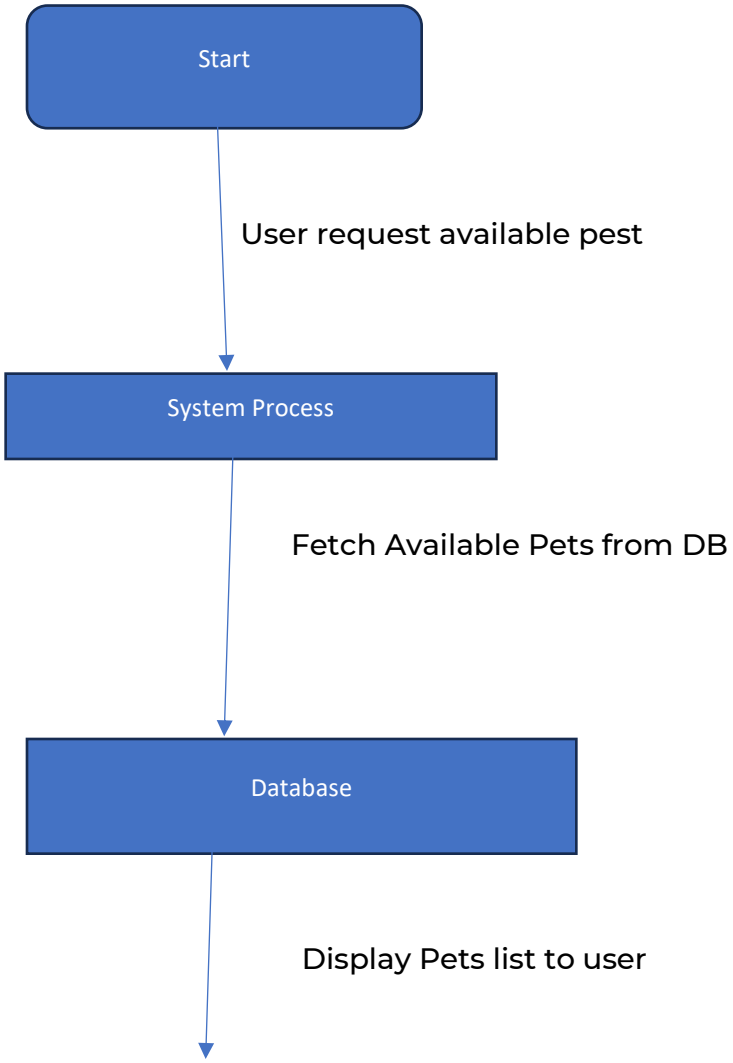


FIGURE 11

2.3.6 Payment

The user enters payment details and selects a payment method. The system processes the payment and stores the transaction in the payment database. The system then sends a payment confirmation or failure message to the user.

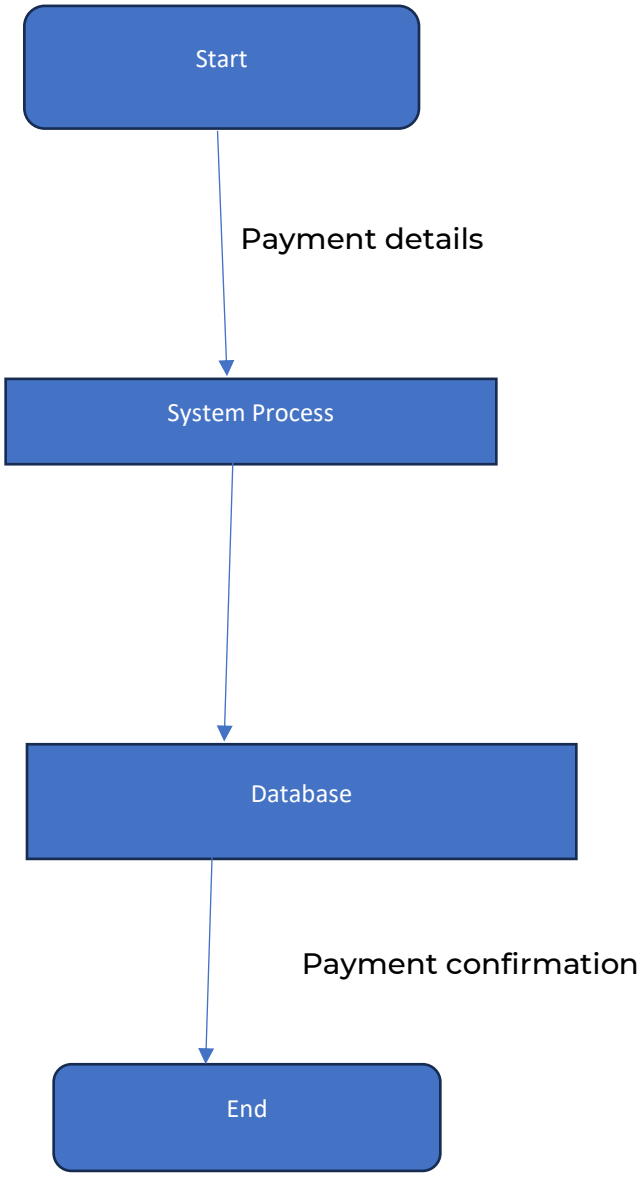
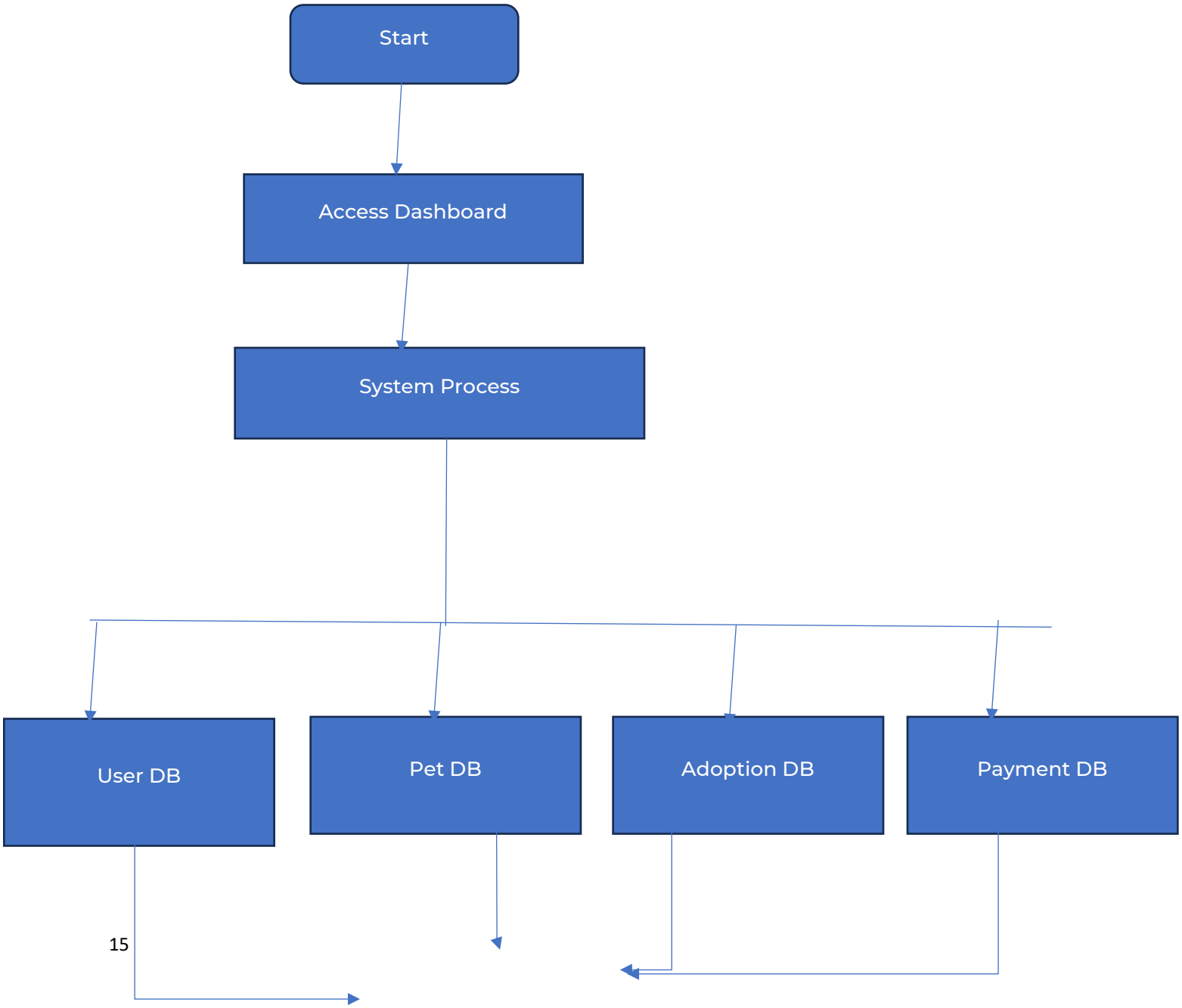


FIGURE 12

2.3.7 ADMIN DASHBOARD

The admin sends management requests to the system . The system retrieves and updates data from the user, pet , adoption and payment databases. The system then displays system reports and statistics to the admin.



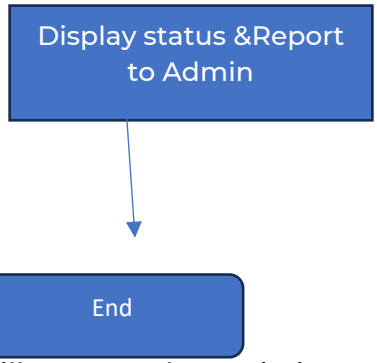


FIGURE 13

2.3.8 Use case diagra

The use case diagram illustrates the main interactions between users and the Pet Adoption System. The system has two actors: the Adopter and the Administrator. This diagram shows the core functionalities of the system and how each user interacts with them.

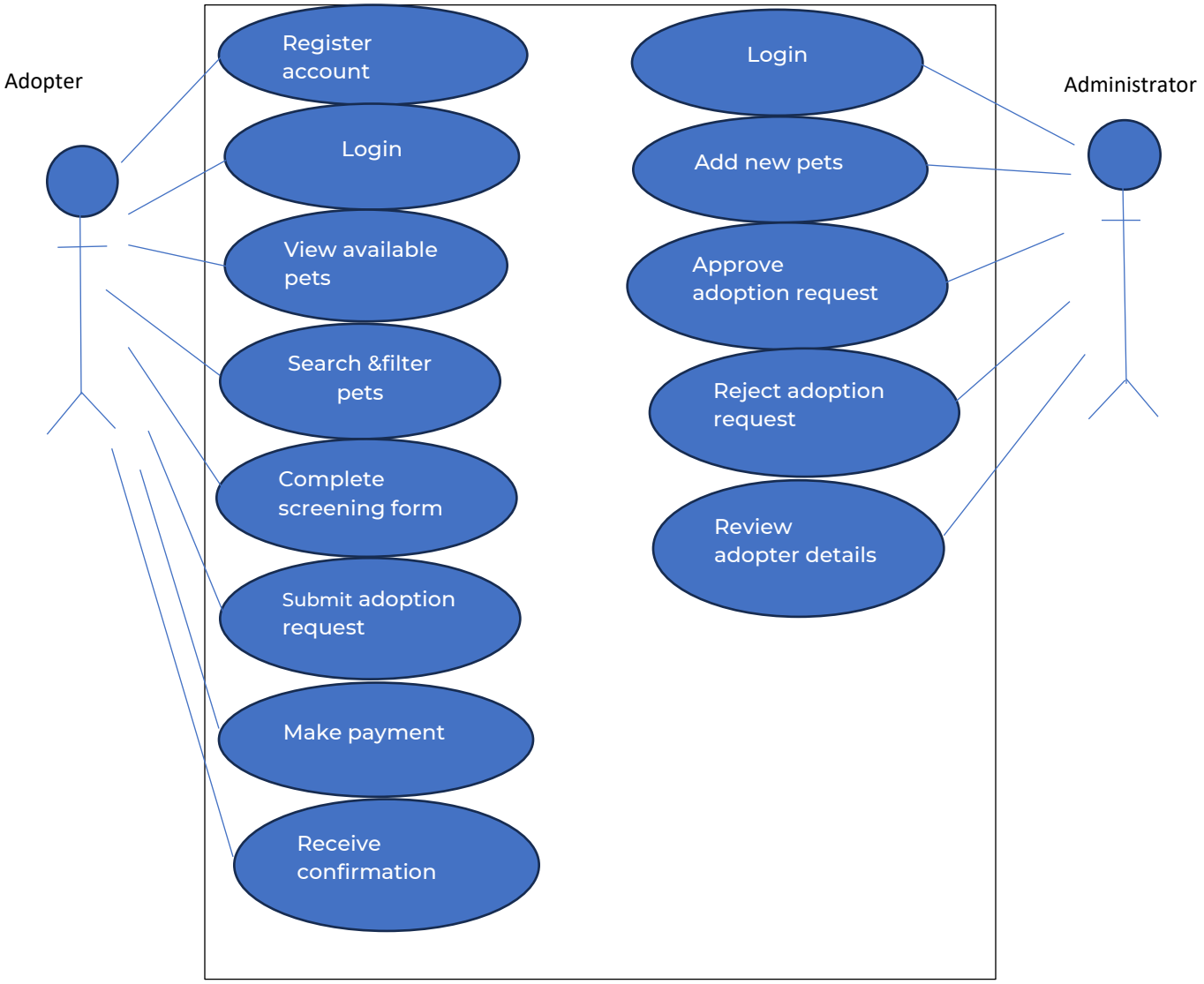


FIGURE 15

2.3.9 Entity-Relationship Diagram(ERD)

The ERD shows how users, pets and adoption requests are related. It includes three main entities, USER which stores user information, PET which stores details about the animals and ADOPTION REQUEST which acts as a linking entity between users and pets. The diagram illustrates that one user can make many adoption requests and one pet can receive many requests, while each adoption request is associated with exactly one user and one pet.

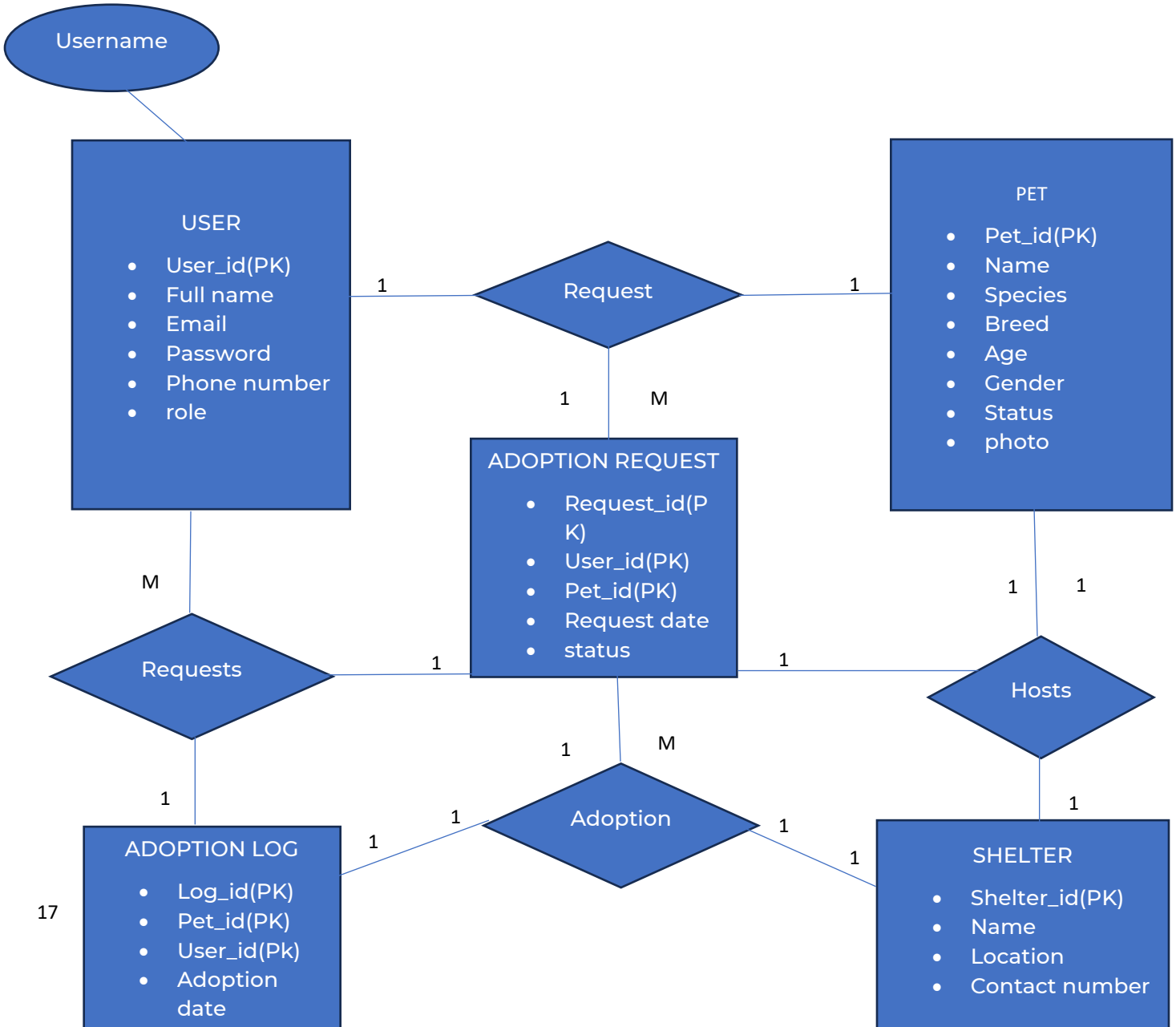
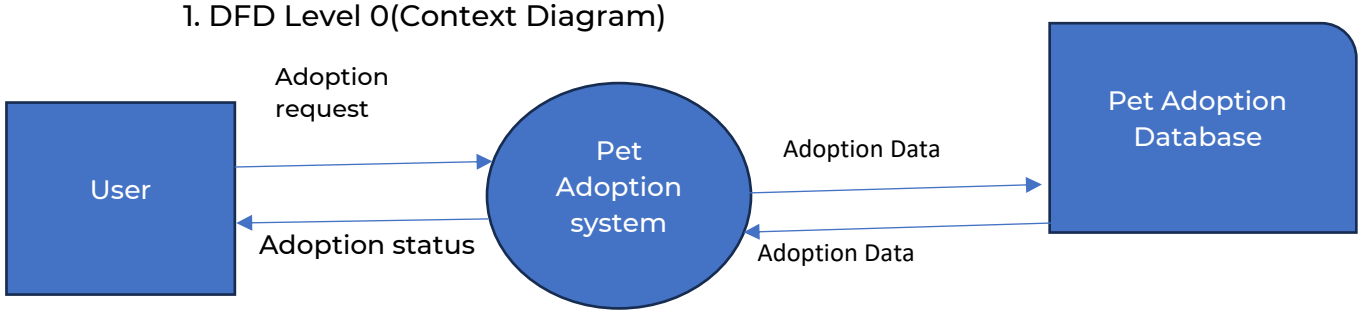


FIGURE 15

2.3.10 Data Flow Diagrams(DFD)

1. DFD Level 0(Context Diagram)



2. DFD Level 1(Detailed diagram)

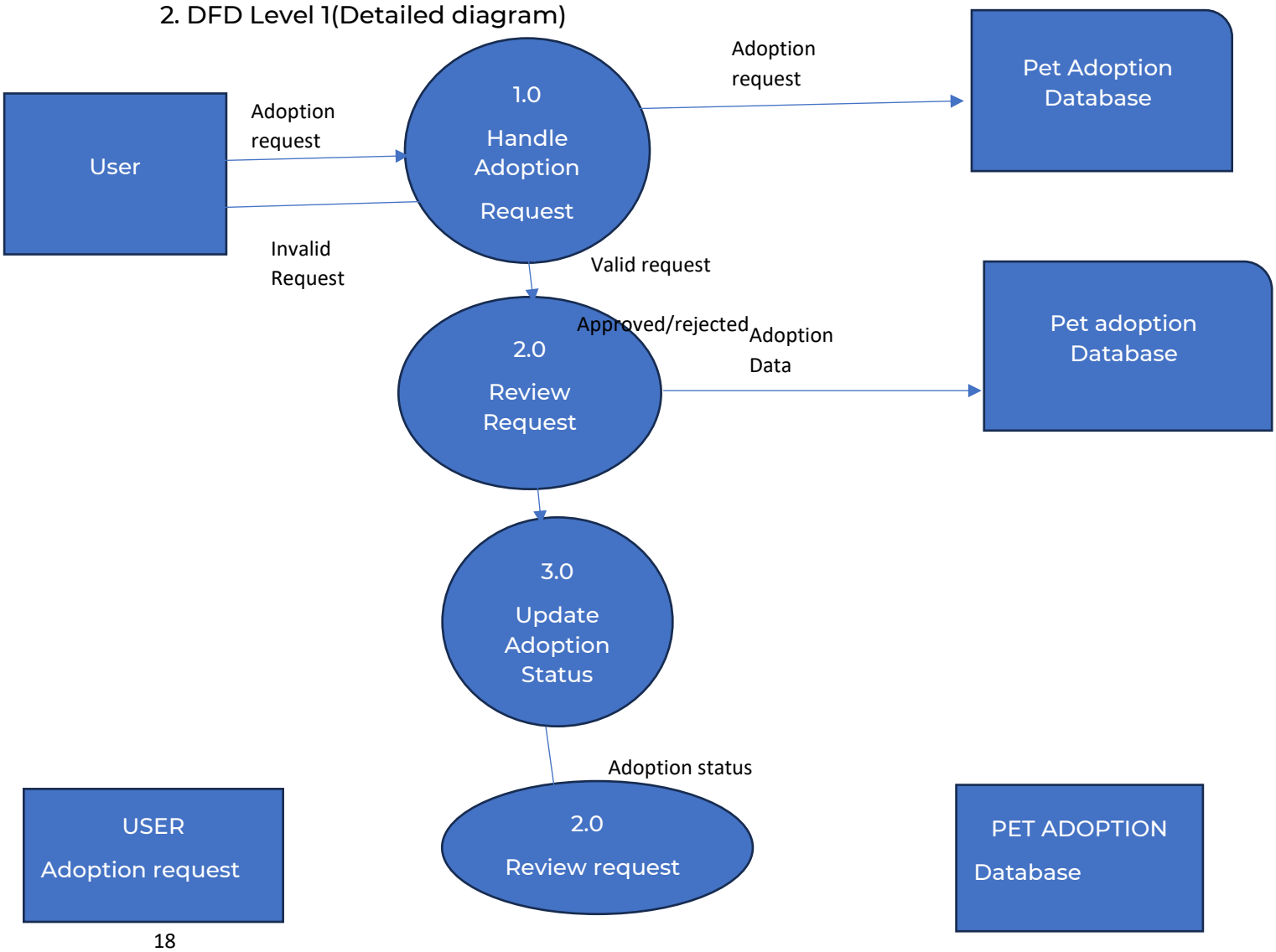


FIGURE 16

FIGURE 17

CHAPTER THREE: SYSTEM IMPLEMENTATION (DEVELOPMENT, TESTING AND DEPLOYMENT)

This is where you report on your development journey e.g. from when you downloaded your IDEs, and coding, to implementing your database to deploying your system

3.1 Introduction

This chapter explains the process followed during the development, testing and deployment of the Pet Adoption System. It covers the creation of the user interface, the system logic that processes data, the testing of features and how the system was made available online.

3.2 User Interface Development

The user interface is the part of the system users interact with. The system was built to be simple and easy to use. The UI includes login and registration pages, pet details pages, pet listing pages , screening form and an admin dashboard. It was developed using React.js, HTML and CSS.

3.2.1 Login Page

The login page allows users to access the system using their email and password.

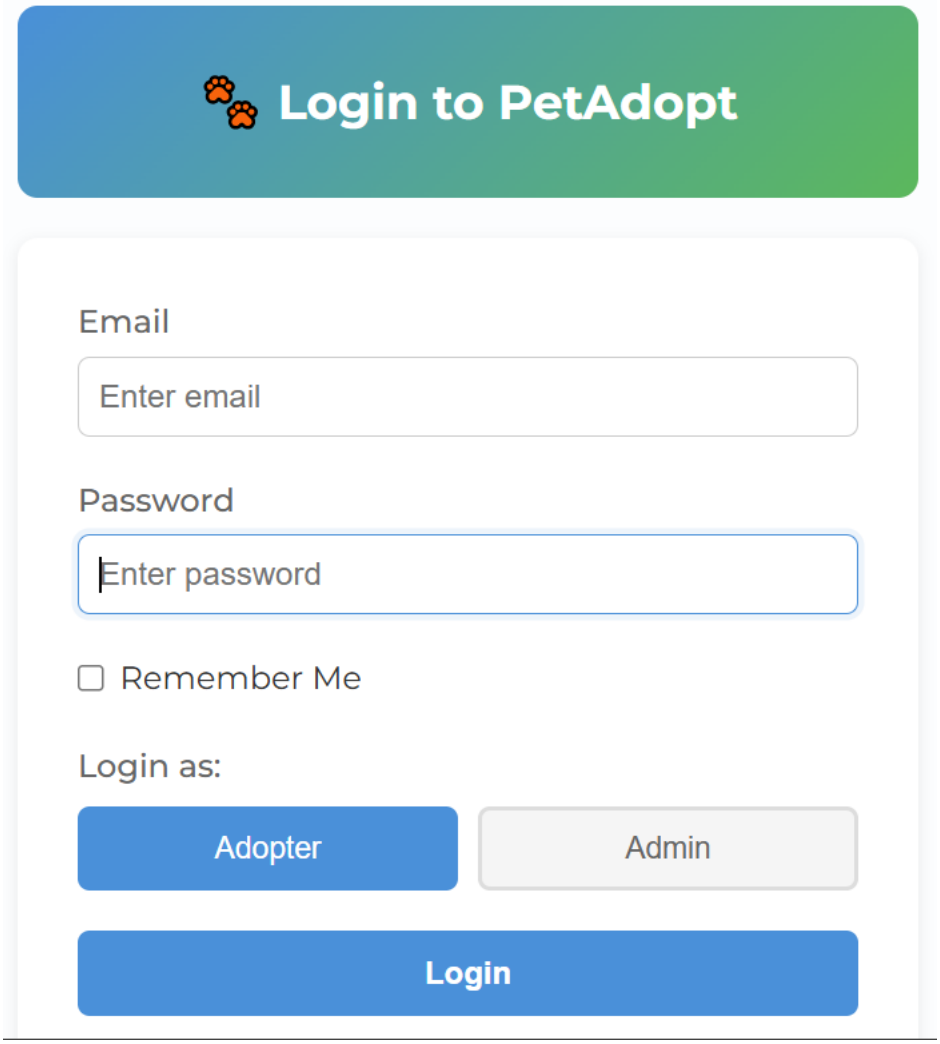


Figure 3.1 Login Page

pet-project-react > src > components > login.jsx > Login

```jsx
import React, { useState, useContext } from "react";
import { Link, useNavigate } from "react-router-dom";
import { AuthContext } from "../context/AuthContext";
import "../assets/login.css";
import apiService from "../services/api";

export default function Login() {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [rememberMe, setRememberMe] = useState(false);
  const [error, setError] = useState("");
  const [loading, setLoading] = useState(false);
  const [role, setRole] = useState("adopter");

  const { login } = useContext(AuthContext);
  const navigate = useNavigate();

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError("");
    setLoading(true);

    if (!email || !password) {
      setError("Please enter both email and password");
      setLoading(false);
      return;
    }

    try {
      // Admin login (local credentials)
```

```jsx
      // Admin login (local credentials)
      if (role === "admin") {
        if (email === "admin@petadopt.com" && password === "admin123") {
          const adminUser = {
            id: 0,
            name: "Admin User",
            full_name: "Administrator",
            email,
            role: "admin",
            token: null
          };
          login(adminUser);
          navigate("/admin");
          return;
        } else {
          setError("User not found");
          setLoading(false);
          return;
        }
      }

      // Adopter login via API
      const response = await apiService.login({ email, password });

      if (response.success) {
        const userData = {
          id: response.user?.id,
```

```
57            name: response.user?.name || "",
58            email: response.user?.email || email,
59            full_name: response.user?.name || "",
60            phone: response.user?.phone || "",
61            role: response.user?.role || "adopter",
62            token: response.token
63          };
64
65          // Use context login to store user and token
66          login(userData);
67          navigate("/pets");
68        } else {
69          setError(response.message || "Login failed. Check your credentials.");
70        }
71      } catch (err) {
72        console.error("Login error:", err);
73        setError("Failed to connect to server. Please try again.");
74      } finally {
75        setLoading(false);
76      }
77    };
78
79    return (
80      <div className="Login">
81        <div className="header">
82          <h2>🐾 Login to PetAdopt</h2>
```

```
83        </div>
84
85        <div className="container">
86          <form onSubmit={handleSubmit}>
87            {error && <div className="error-message">{error}</div>}
88
89            <div className="form-group">
90              <label>Email</label>
91              <input
92                type="email"
93                placeholder="Enter email"
94                value={email}
95                onChange={(e) => setEmail(e.target.value)}
96                required
97              />
98            </div>
99
100           <div className="form-group">
101             <label>Password</label>
102             <input
103               type="password"
104               placeholder="Enter password"
105               value={password}
106               onChange={(e) => setPassword(e.target.value)}
107               required
108             />
109           </div>
110
111           <div className="checkbox">
```

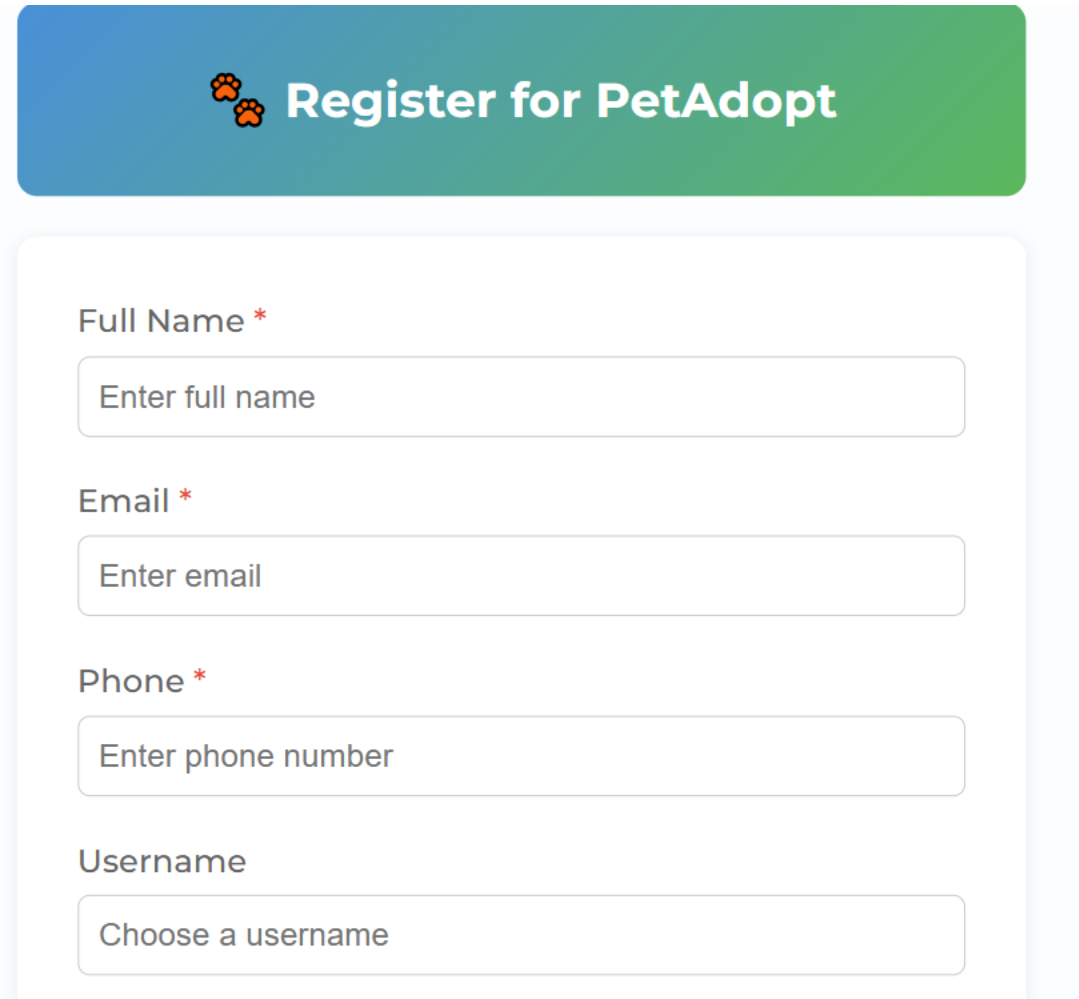
```
112                <input
113                  type="checkbox"
114                  id="rememberMe"
115                  checked={rememberMe}
116                  onChange={(e) => setRememberMe(e.target.checked)}
117                />
118                <label htmlFor="rememberMe">Remember Me</label>
119              </div>
120
121              <div className="role-selection">
122                <label>Login as:</label>
123                <div className="role-buttons">
124                  <button
125                    type="button"
126                    className={`role-btn ${role === "adopter" ? "active" : ""}`}
127                    onClick={() => setRole("adopter")}
128                  >
129                    Adopter
130                  </button>
131                  <button
132                    type="button"
133                    className={`role-btn ${role === "admin" ? "active" : ""}`}
134                    onClick={() => setRole("admin")}
135                  >
136                    Admin
137                  </button>
138                </div>
139              </div>
```

```
141              <button type="submit" disabled={loading}>
142                {loading ? "Logging in..." : "Login"}
143              </button>
144
145              <div className="links">
146                <Link to="/forgot-password">Forgot Password?</Link>
147                <p>
148                  New user? <Link to="/register">Register</Link>
149                </p>
150              </div>
151            </form>
152          </div>
153        </div>
154      );
155    }
156
157
```

Figure 3.2 HTML/React code for login page

3.2.2 Registration Page

The registration page lets new users create an account. User information is validated before being stored in the database.



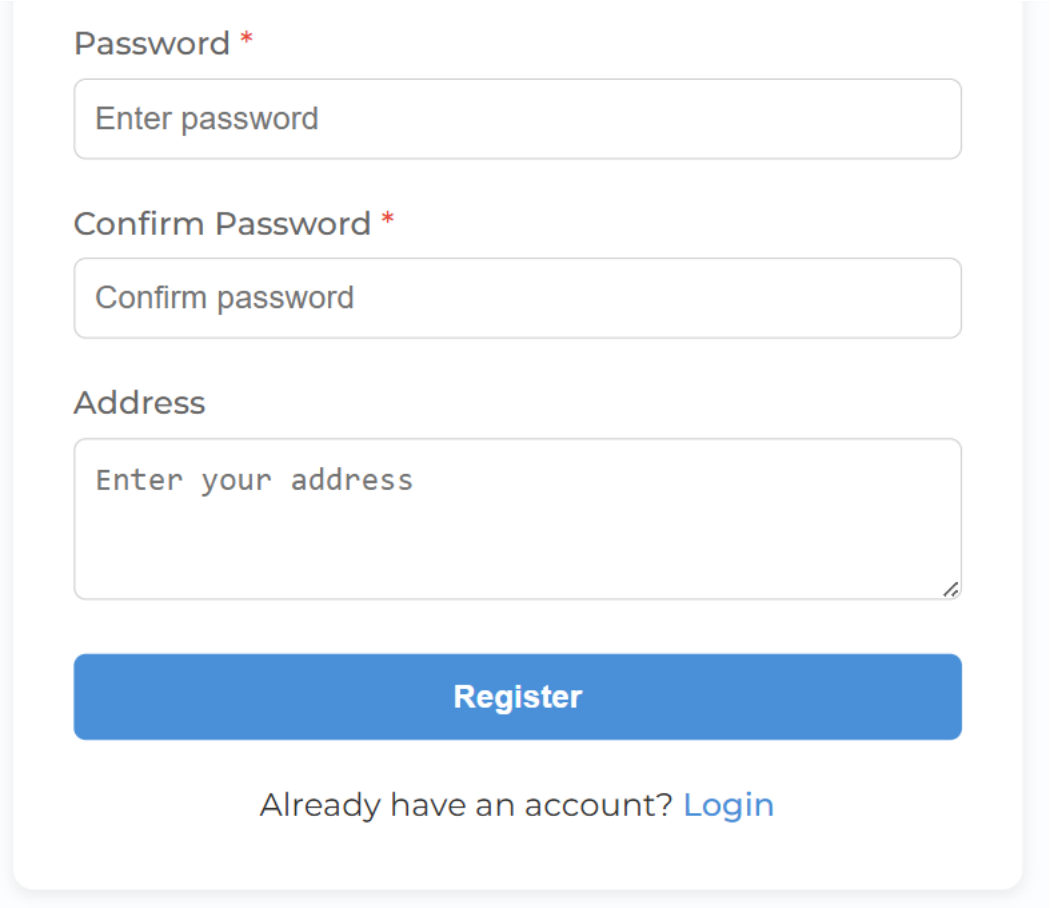


Figure 3.3 Registration form

pet-project-react > src > components > register.jsx > Register

```jsx
import React, { useState } from "react";
import { Link, useNavigate } from "react-router-dom";
import "../assets/register.css";
import apiService from "../services/api";

export default function Register() {
  const [fullName, setFullName] = useState("");
  const [email, setEmail] = useState("");
  const [phone, setPhone] = useState("");
  const [username, setUsername] = useState("");
  const [password, setPassword] = useState("");
  const [confirmPassword, setConfirmPassword] = useState("");
  const [address, setAddress] = useState("");
  const [error, setError] = useState("");
  const [loading, setLoading] = useState(false);

  const navigate = useNavigate();

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError("");
    setLoading(true);

    // Validate passwords match
    if (password !== confirmPassword) {
      setError("Passwords do not match");
      setLoading(false);
      return;
    }

```

```jsx
    // Validate password length
    if (password.length < 6) {
      setError("Password must be at least 6 characters");
      setLoading(false);
      return;
    }

    try {
      console.log("Attempting to register with:", { name: fullName, email, phone });

      const response = await apiService.register({
        name: fullName,
        email: email,
        phone: phone,
        username: username,
        password: password,
        address: address
      });

      console.log("Registration response:", response);

      if (response.success) {
        // Redirect to login after successful registration
        alert("Registration successful! Please login.");
        navigate("/login");
      } else {
```

```
57          setError(response.message || "Registration failed");
58        }
59      } catch (err) {
60        console.error("Registration error:", err);
61        setError("Failed to connect to server. Please check if the backend is running.");
62      } finally {
63        setLoading(false);
64      }
65    };
66
67    return (
68      <div className="register">
69        <div className="header">
70          <h2>🐾 Register for PetAdopt</h2>
71        </div>
72        <div className="container">
73          <form onSubmit={handleSubmit}>
74            {error && <div className="error-message">{error}</div>}
75
76            <div className="form-group">
77              <label>Full Name <span>*</span></label>
78              <input
79                type="text"
80                placeholder="Enter full name"
81                value={fullName}
```

```
82                onChange={(e) => setFullName(e.target.value)}
83                required
84              />
85            </div>
86
87            <div className="form-group">
88              <label>Email <span>*</span></label>
89              <input
90                type="email"
91                placeholder="Enter email"
92                value={email}
93                onChange={(e) => setEmail(e.target.value)}
94                required
95              />
96            </div>
97
98            <div className="form-group">
99              <label>Phone <span>*</span></label>
100             <input
101               type="tel"
102               placeholder="Enter phone number"
103               value={phone}
104               onChange={(e) => setPhone(e.target.value)}
105               required
106             />
107           </div>
108
```

```
109            <div className="form-group">
110              <label>Username</label>
111              <input
112                type="text"
113                placeholder="Choose a username"
114                value={username}
115                onChange={(e) => setUsername(e.target.value)}
116              />
117            </div>
118
119            <div className="form-group">
120              <label>Password <span>*</span></label>
121              <input
122                type="password"
123                placeholder="Enter password"
124                value={password}
125                onChange={(e) => setPassword(e.target.value)}
126                required
127              />
128            </div>
129
130            <div className="form-group">
131              <label>Confirm Password <span>*</span></label>
132              <input
133                type="password"
134                placeholder="Confirm password"
135                value={confirmPassword}
136                onChange={(e) => setConfirmPassword(e.target.value)}
137                required
138              />
139            </div>
140
141            <div className="form-group">
142              <label>Address</label>
143              <textarea
144                placeholder="Enter your address"
145                value={address}
146                onChange={(e) => setAddress(e.target.value)}
147              ></textarea>
148            </div>
149
150            <button type="submit" disabled={loading}>
151              {loading ? "Registering..." : "Register"}
152            </button>
153
154            <p className="login-link">
155              Already have an account? <Link to="/login">Login</Link>
156            </p>
157          </form>
158        </div>
159      </div>
160    );
161  }
```

Figure 3.4 Code for registration form

3.2.3 Pet listing page

This page shows the available pets for adoption and users can browse and view pet details.

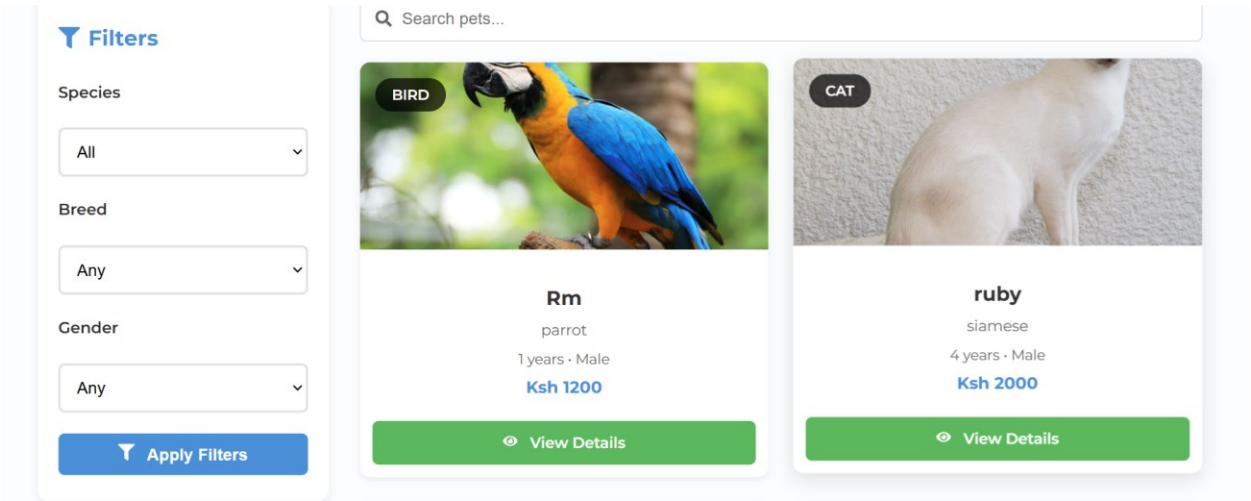


Figure 3.5 pet listing page

```
import { Link } from "react-router-dom";
import { FaSearch, FaFilter, FaEye } from "react-icons/fa";
import "../assets/petlist.css";
import apiService from "../services/api";

function Petlist() {
  const [pets, setPets] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  const [searchTerm, setSearchTerm] = useState("");
  const [filters, setFilters] = useState({
    species: "",
    breed: "",
    gender: ""
  });

  const dogBreeds = ["Golden Retriever", "Labrador", "German Shepherd", "Bulldog", "B
  const catBreeds = ["Persian", "Siamese", "Maine Coon", "British Shorthair", "Ragdol
  const birdBreeds = ["Parrot", "Canary", "Sparrow", "Finch"];
  const rabbitBreeds = ["Holland Lop", "Netherland Dwarf", "Lionhead", "Chinchilla"];

  useEffect(() => {
    fetchPets();
  }, []); // initial load

  const fetchPets = async () => {
    setLoading(true);
    setError(null);
    try {
```

Figure 3.6 code for petlist

3.2.4 Pet Details Page

Shows detailed information about a selected pet, including gender, color, breed, age, health status, temperament and price.

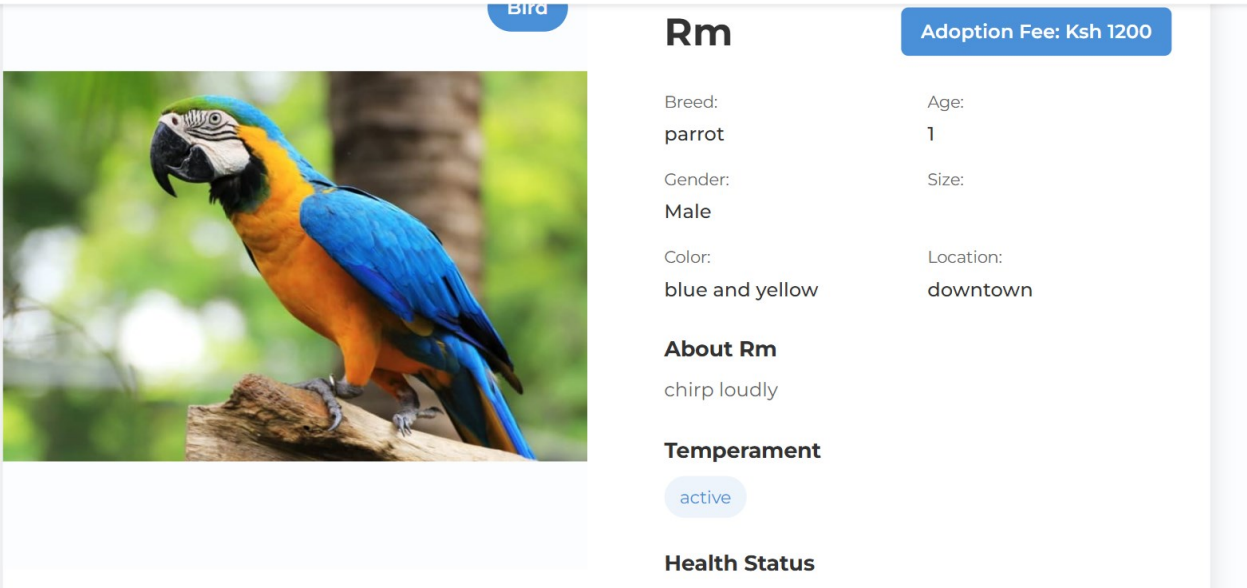


Figure 3.7 Pet details page

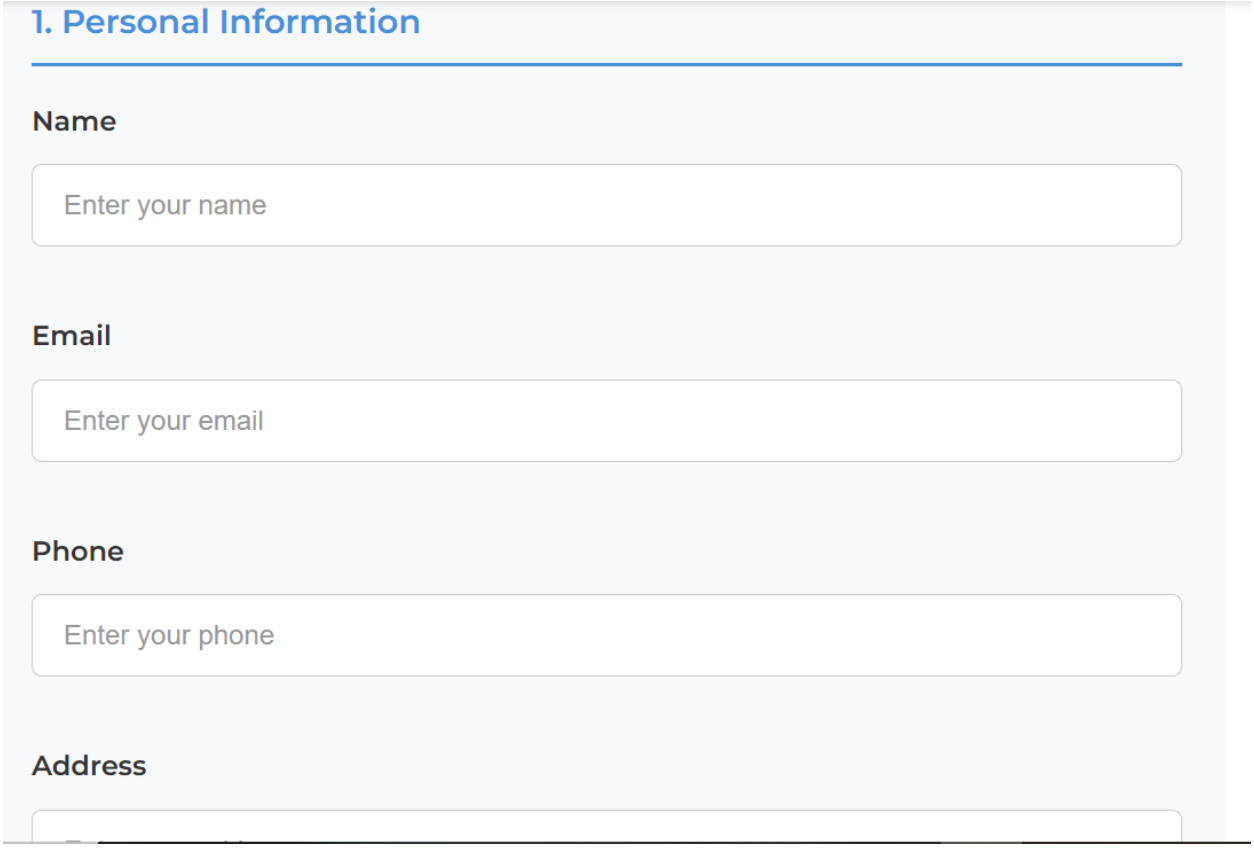
```
import React, { useState, useEffect } from "react";
import { useParams, Link } from "react-router-dom";
import { FaArrowLeft, FaHeart } from "react-icons/fa";
import "../assets/petdetails.css";
import apiService from "../services/api";

function PetDetails() {
  const { id } = useParams();
  const [pet, setPet] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");

  useEffect(() => {
    const fetchPet = async () => {
      setLoading(true);
      setError("");
      try {
        const data = await apiService.getPetById(id); // fetch single pet from API
        if (data) {
          setPet(data);
        } else {
          setError("Pet not found");
        }
      } catch (err) {
        console.error("Error fetching pet details:", err);
        setError("Failed to load pet details. Please try again.");
      } finally {
        setLoading(false);
      }
    };
```

```
  fetchPet();
}, [id]);

const getDefaultImage = (type) => {
  if (type === "Cat") return "https://images.unsplash.com/photo-1573865526739-10659
  if (type === "Dog") return "https://images.unsplash.com/photo-1552053831-71594a27
  return "https://images.unsplash.com/photo-1508672019048-805c876b67e2?w=400&h=3008
};

if (loading) {
  return <div className="pet-details-container"><p>Loading pet details...</p></div>
}

if (error || !pet) {
  return (
    <div className="pet-details-container">
      <h2>{error || "Pet not found"}</h2>
      <Link to="/pets" className="btn-primary">
        Back to Pet List
      </Link>
    </div>
  );
}

return (
  <div className="pet-details-container">
    <div class="pet-details-wrapper">
```

Figure 3.8 code for pet details

3.2.5 Screening form

Adopters filling the form for verification before adopting a pet.



2. Living Situation

Residence

Select Residence

Fenced Yard

Fenced Yard

3. Pet Experience

Pet Experience

4. Travel Care

Travel Care

Who will care for pet when traveling

5. Guarantor Information

Guarantor Name

Enter guarantor name

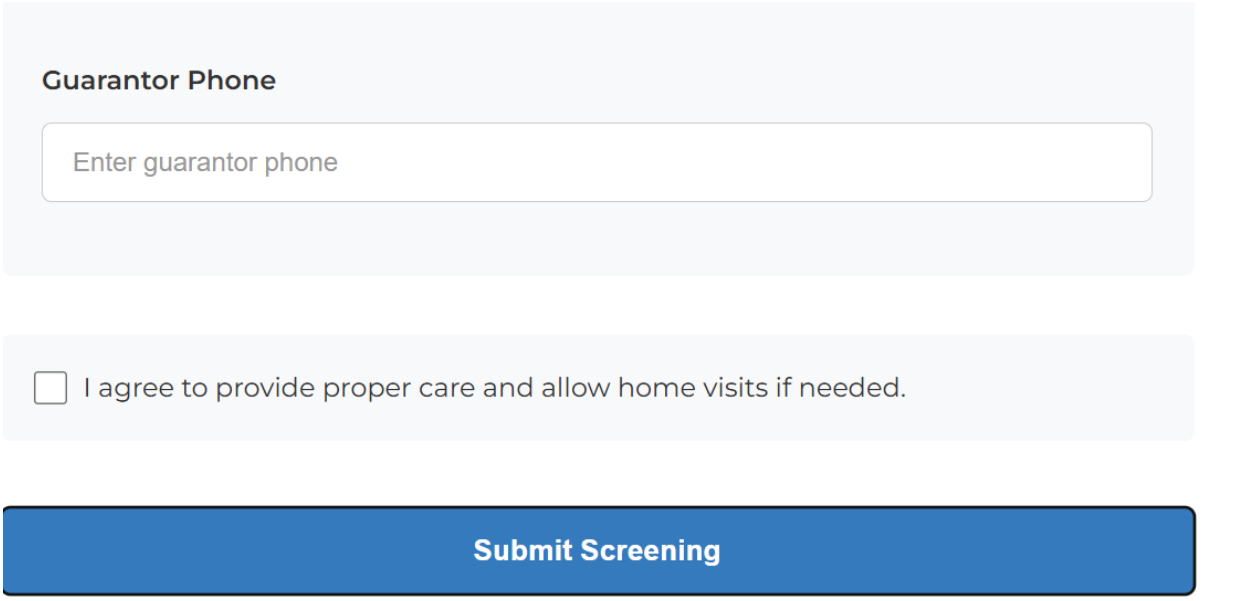


Figure 3.9 Screening form page

```
1   import React, { useState, useEffect } from "react";
2   import { useLocation, useNavigate, useParams } from "react-router-dom";
3   import apiService from "../services/api";
4
5   const Screening = () => {
6     const location = useLocation();
7     const navigate = useNavigate();
8     const params = useParams();
9     const petId = location.state?.petId || params.petId;
10
11    const user = JSON.parse(localStorage.getItem("user"));
12
13    const [formData, setFormData] = useState({
14      phone: "",
15      residence: "",
16      fencedYard: "",
17      ownedPets: "",
18      experience: "",
19      aloneHours: "",
20      travelCare: "",
21      guarantorName: "",
22      guarantorPhone: "",
23      agreement: false
24    });
25
26    const handleChange = (e) => {
27      const { name, value, type, checked } = e.target;
28      setFormData({
29        ...formData,
30        [name]: type === "checkbox" ? checked : value
```

```
34  const handleSubmit = async (e) => {
35    e.preventDefault();
36
37    if (!petId) {
38      alert("No pet selected.");
39      navigate("/pets");
40      return;
41    }
42
43    if (!formData.agreement) {
44      alert("You must agree to terms.");
45      return;
46    }
47
48    if (!user?.id) {
49      alert("You must be logged in.");
50      return;
51    }
52
53    const requestData = {
54      user_id: user.id,
55      pet_id: petId,
56      fullName: user.name || "",
57      email: user.email || "",
58      phone: formData.phone,
59      residence: formData.residence,
60      fencedYard: formData.fencedYard,
61      ownedPets: formData.ownedPets,
```

Figure 3.10 code for screening form

3.2.6 Admin dashboard

Allows the administrator to manage pets and users, add new pets, and monitor adoption requests.

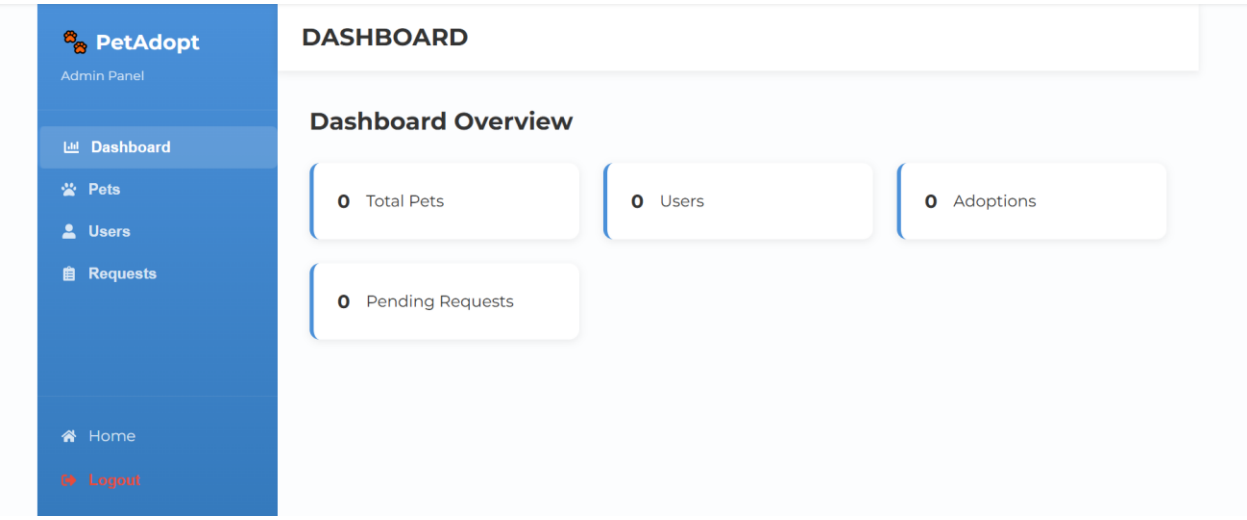


Figure 3.11 Admin dashboard page

```
import React, { useState, useEffect, useContext } from "react";
import { Link, useNavigate } from "react-router-dom";
import {
  FaPaw,
  FaUsers,
  FaHome,
  FaClipboardList,
  FaChartBar,
  FaSignOutAlt,
  FaPlus,
  FaEdit,
  FaTrash,
  FaCheck,
  FaTimes,
  FaUser
} from "react-icons/fa";
import { AuthContext } from "../context/AuthContext";
import "../assets/admin.css";
import apiService from "../services/api";

function Admin() {
  const [activeTab, setActiveTab] = useState("dashboard");
  const [loading, setLoading] = useState(false);
  const { logout } = useContext(AuthContext);
  const navigate = useNavigate();


  useEffect(() => {
    const user = JSON.parse(localStorage.getItem("user") || "{}");
```

```
    if (!user || user.role !== "admin") {
      navigate("/login");
    }
  }, [navigate]);

  const [pets, setPets] = useState([]);
  const [users, setUsers] = useState([]);
  const [adoptionRequests, setAdoptionRequests] = useState([]);
  const [stats, setStats] = useState({
    totalPets: 0,
    activeUsers: 0,
    adoptions: 0,
    pendingRequests: 0
  });

  const [showDeleteModal, setShowDeleteModal] = useState(false);
  const [petToDelete, setPetToDelete] = useState(null);

  useEffect(() => {
    if (activeTab === "dashboard" || activeTab === "requests") fetchApplications();
    if (activeTab === "pets") fetchPets();
    if (activeTab === "users") fetchUsers();
  }, [activeTab]);


  const fetchPets = async () => {
    setLoading(true);
```

3.3 Logic Development

This section explains how the system processes data and performs different operations such as login validation, storing user data, retrieving pet information, and handling adoption requests. The logic is implemented using PHP and MySQL to ensure the system functions correctly.

3.3.1 Login validation

This logic checks whether the email and password entered by the user match the records stored in the database. If the details are correct, the user is allowed to access the system. If not an error message is displayed.

```php
<?php
require_once __DIR__ . '/../cors.php';
require_once __DIR__ . '/../../config/db.php';


$data = json_decode(file_get_contents("php://input"), true);

$email = $data['email'] ?? '';
$password = $data['password'] ?? '';
$requested_role = $data['role'] ?? 'adopter';

if (empty($email) || empty($password)) {
    echo json_encode(["success" => false, "message" => "Email and password required"]
    exit();
}


$stmt = $conn->prepare("SELECT * FROM users WHERE email = ?");
$stmt->execute([$email]);
$user = $stmt->fetch(PDO::FETCH_ASSOC);

if (!$user) {
    echo json_encode(["success" => false, "message" => "User not found"]);
    exit();
}

if (!password_verify($password, $user['password'])) {
    echo json_encode(["success" => false, "message" => "Invalid password"]);
    exit();
}
```

```php

if ($user['role'] !== $requested_role) {
    echo json_encode(["success" => false, "message" => "This account cannot login as
    exit();
}


$token = bin2hex(random_bytes(32));

echo json_encode([
    "success" => true,
    "message" => "Login successful",
    "token" => $token,
    "user" => [
        "id" => $user['id'],
        "name" => $user['name'],
        "email" => $user['email'],
        "role" => $user['role']
    ]
]);
```

Figure 3.12 PHP code for login validation

3.3.2 Registration validation

This logic handles the registration process by collecting user details from the form and storing them in the database. It ensures that all required fields are filled before saving the data.

```php
<?php

require_once __DIR__ . '/../cors.php';
require_once __DIR__ . '/../../config/db.php';

$data = json_decode(file_get_contents("php://input"), true);
$name = trim($data['name'] ?? '');
$email = trim($data['email'] ?? '');
$password = trim($data['password'] ?? '');
if (empty($name) || empty($email) || empty($password)) {
    echo json_encode(["success" => false, "message" => "All fields are required"]);
    exit();
}
$stmt = $conn->prepare("SELECT id FROM users WHERE email = ?");
$stmt->execute([$email]);

if ($stmt->rowCount() > 0) {
    echo json_encode(["success" => false, "message" => "Email already registered"]);
    exit();
}
$hashedPassword = password_hash($password, PASSWORD_DEFAULT);
$stmt = $conn->prepare("INSERT INTO users (name, email, password, role) VALUES (?, ?,
$stmt->execute([$name, $email, $hashedPassword]);
if ($stmt->rowCount() > 0) {
    echo json_encode(["success" => true, "message" => "Registration successful"]);
} else {
    echo json_encode(["success" => false, "message" => "Registration failed"]);
}
?>
```

Figure 3.13 PHP code for registration validation

3.3.3 Pet Upload

This logic allows admin to add new pets to the system soring details like breed, name, age and image in the database.

```php
<?php
require_once __DIR__ . '/../cors.php';
require_once __DIR__ . '/../../config/db.php';

$response = ["success" => false, "message" => ""];

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $name = $_POST['name'] ?? '';
    $species = $_POST['species'] ?? '';
    $breed = $_POST['breed'] ?? '';
    $age = $_POST['age'] ?? '';
    $gender = $_POST['gender'] ?? '';
    $description = $_POST['description'] ?? '';
    $status = $_POST['status'] ?? 'available';
    $color = $_POST['color'] ?? '';
    $temperament = $_POST['temperament'] ?? '';
    $health = $_POST['health'] ?? '';
    $location = $_POST['location'] ?? '';
    $adoption_fee = $_POST['adoption_fee'] ?? 0;

    $image = '';
    $uploaded_file_path = '';
    if (isset($_FILES['image']) && $_FILES['image']['error'] === 0) {
        $img_name = time() . '_' . basename($_FILES['image']['name']);
        $target_dir = '../../uploads/';
        $target_file = $target_dir . $img_name;
        $allowed_types = ['image/jpeg', 'image/png', 'image/gif', 'image/webp'];
        $file_type = $_FILES['image']['type'];

        if (!in_array($file_type, $allowed_types)) {
```

```php
            $response['message'] = "Invalid image type. Allowed types: JPEG, PNG, GIF
            echo json_encode($response);
            exit;
        }
        if ($_FILES['image']['size'] > 5 * 1024 * 1024) {
            $response['message'] = "Image size too large. Maximum size is 5MB.";
            echo json_encode($response);
            exit;
        }
        if (!move_uploaded_file($_FILES['image']['tmp_name'], $target_file)) {
            $response['message'] = "Failed to upload image.";
            echo json_encode($response);
            exit;
        }
        $image = $img_name;
        $uploaded_file_path = $target_file;
    } else {
        $image = '';
    }
    $sql = "INSERT INTO pets
        (name, species, breed, age, gender, description, image, status, color, temper
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)";
    $params = [$name, $species, $breed, $age, $gender, $description, $image, $status,
    $stmt = $conn->prepare($sql);
    if ($stmt->execute($params)) {
        $response['success'] = true;
        $response['message'] = "Pet added successfully";
    } else {

        if (!empty($uploaded_file_path) && file_exists($uploaded_file_path)) {
```

Figure 3.14 PHP code for pet upload

3.3.4 Screening form processing

This logic collects information entered by the adopter in the screening form and stores it in the database. The data is then used by the administrator to review the adoption request.

```php
<?php
require_once __DIR__ . '/../cors.php';
require_once __DIR__ . '/../../config/db.php';
$status = isset($_GET['status']) ? $_GET['status'] : 'pending';
try {
    $sql = "SELECT
        sf.id,
        sf.user_id,
        sf.pet_id,
        sf.name,
        sf.email,
        sf.phone,
        sf.residence,
        sf.fenced_yard,
        sf.pet_experience,
        sf.hours_alone,
        sf.travel_care,
        sf.guarantor_name,
        sf.guarantor_phone,
        sf.status,
        sf.created_at,
        u.name as user_name,
        u.email as user_email,
        p.name as pet_name,
        p.image as pet_image,
        p.adoption_fee
    FROM screening_forms sf
    LEFT JOIN users u ON sf.user_id = u.id
    LEFT JOIN pets p ON sf.pet_id = p.id
    WHERE sf.status = :status
```

```php
$applications = [];
while ($req = $stmt->fetch(PDO::FETCH_ASSOC)) {
    $petImage = !empty($req['pet_image'])
        ? 'http://localhost/pet%20project/petprojectt/uploads/' . $req['pet_image
        : 'http://localhost/pet%20project/petprojectt/uploads/imageee.jpg';
    $applications[] = [
        'id' => $req['id'],
        'screeningId' => $req['id'],
        'petId' => $req['pet_id'],
        'petName' => $req['pet_name'] ?? 'Unknown Pet',
        'petImage' => $petImage,
        'adoptionFee' => $req['adoption_fee'] ?? 0,
        'userId' => $req['user_id'],
        'fullName' => $req['name'],
        'email' => $req['email'],
        'phone' => $req['phone'] ?? '',
        'residence' => $req['residence'] ?? '',
        'fencedYard' => $req['fenced_yard'] ?? '',
        'petExperience' => $req['pet_experience'] ?? '',
        'hoursAlone' => $req['hours_alone'] ?? '',
        'travelCare' => $req['travel_care'] ?? '',
        'guarantorName' => $req['guarantor_name'] ?? '',
        'guarantorPhone' => $req['guarantor_phone'] ?? '',
        'status' => $req['status'],
        'date' => $req['created_at']
    ];
}
echo json_encode([
    "success" => true,
    "applications" => $applications
```

Figure 3.15 PHP code for screening form processing

3.3.5 Admin approval

This logic allows the administrator to review adoption requests and update their status. The admin can approve or reject a request and the system updates the database accordingly.

```php
<?php
require_once __DIR__ . '/../api/cors.php';
include __DIR__ . '/../config/mailer.php';
require_once __DIR__ . '/../config/db.php';
$data = json_decode(file_get_contents("php://input"), true);
if (!isset($data['request_id'])) {
    echo json_encode(["success" => false, "message" => "Request ID required"]);
    exit;
}
$request_id = intval($data['request_id']);
$stmt = $conn->prepare("
    SELECT ar.id, u.email, u.name, p.name AS pet_name
    FROM adoption_requests ar
    JOIN users u ON ar.user_id = u.id
    JOIN pets p ON ar.pet_id = p.id
    WHERE ar.id = ?
");
$stmt->execute([$request_id]);
$request = $stmt->fetch(PDO::FETCH_ASSOC);

if (!$request) {
    echo json_encode(["success" => false, "message" => "Request not found"]);
    exit;
}
$email = $request['email'];
$name = $request['name'];
$pet_name = $request['pet_name'];
$update = $conn->prepare("UPDATE adoption_requests SET request_status = 'approved' WH
$update->execute([$request_id]);
$payment_link = "http://localhost:5173/payment?request_id=" . $request_id;
```

```php
if ($emailSent) {
    error_log("Approval email sent successfully to: $email");
} else {
    error_log("FAILED to send approval email to: $email. Check PHP mail configuratior
}

echo json_encode([
    "success" => true,
    "message" => "Adoption request approved",
    "email_sent" => $emailSent,
    "email_note" => $emailSent ? "Approval email sent to adopter" : "Email could not
]);
```

```php
        if ($update->rowCount() > 0) {

            $emailSent = sendRejectionEmail($email, $name, $pet_name);
            if ($emailSent) {
                error_log("Rejection email sent to: $email");
            } else {
                error_log("FAILED to send rejection email to: $email");
            }
            echo json_encode([
                "success" => true,
                "message" => "Adoption request rejected",
                "email_sent" => $emailSent,
                "email_note" => $emailSent ? "Rejection email sent to adopter" : "Email c
            ]);
        } else {
            echo json_encode([
                "success" => false,
                "message" => "Failed to reject request"
            ]);
        }
    } catch (PDOException $e) {
        echo json_encode(["success" => false, "message" => "Database error: " . $e->getMe
    }
```

Figure 3.16 PHP code for admin approval

3.4 Testing

I carried out testing to make sure the system works correctly. I tested login using valid and invalid details to ensure correct users can access the system. And tested registration form to confirm user data is saved properly.

I also tested pet listing and pet details pages to ensure pet information is displayed correctly. Tested screening form to confirm that adopter details are submitted and stored in the database.

Found some errors like incorrect API links and database connection issues and fixed them. After fixing them I tested the system again and it worked as expected.

3.5 Deployment

The system is deployed online using Infinityfree. I uploaded the frontend and backend files to the hosting server and created the database using phpMyAdmin.

I then connected the system to the online database and updated the API links. After deployment , I tested the system to ensure everything was working properly

System URL: https://petadoption.infinityfreeapp.com/

The system can now be accessed by users through web browser.

CHAPTER FOUR: CONCLUSION AND RECOMMENDATION

4.1 Conclusion

This project focused on the design and development of a pet adoption system. Chapter one introduced the project and its objectives. Chapter two covered the system design and modelling, including how the system structure and processes were planned. Chapter three explained how the system was developed , tested and deployed. During the development process, I learned how to connect a React frontend with a PHP backend and a MySQL database. I also gained experience in hosting a system online using Infinityfree. Some challenges I faced include database connection errors, incorrect API URLs and hosting issues but they were resolved through testing and debugging.

4.2 Recommendation

Future developers can enhance the system by implementing a real-time chat support so adopters can communicate directly with administrators for inquiries or guidance during the adoption process. They can also integrate MPESA payment to allow users to pay adoption fee online instead of handling it manually. Additionally , integrating automated reminders and alerts for pending applications or upcoming adoption events would improve user engagement and ensure timely follow-ups.

REFERENCES

Haste, I., & Thakare, T.(2025). Efficient frame work for pet adoption. International students' conference on Electrical, Electronics and computer science.

Dog Welfare project. (2023)

Woodford Humane Society.(2022).The adoption process. https://www.woodfordhumane.org/